

SmartPet AI (V1) - Project Overview

A highly focused, multi-modal telehealth and e-commerce mobile application. V1 strips away complex video streaming and live web-sockets to focus on an API-driven architecture. It features a React Native shell connecting to a Node.js core, with specialized RAG (text), spatial (image), and temporal (video) microservices generating diagnostic data to drive a personalized e-commerce recommendation engine.

Functional Requirements (V1)

- **Pet Profiles:** Users can create, read, update, and delete basic pet data (Breed, Age, Allergies) stored in MongoDB.
- **Triage Chatbot:** A pure text interface that routes queries to an LLM grounded specifically in the active pet's database profile (RAG).
- **Skin/Eye Scanner:** A dedicated UI button allowing users to upload a single `.jpg`, returning a condition and confidence score.
- **Motion Scanner:** A dedicated UI button allowing users to upload a 5-second `.mp4`, returning a basic gait anomaly flag.
- **Smart Storefront:** A product feed that algorithmically filters out allergens and promotes items related to the AI diagnostic results.
- **WhatsApp Checkout:** A frictionless cart that serializes items into a URL string for WhatsApp messaging.

Non-Functional Requirements (V1)

- **Latency:** All AI inference endpoints (RAG, Image, Video) must return a response to the frontend in under 4 seconds.
- **Decoupled Architecture:** ML microservices must remain completely stateless; they receive a file/text, return JSON, and immediately purge the media.
- **Standardization:** All Node.js/React code must adhere to strict explicit functional declarations (no destructuring, standard returns).

Task Delegation (The 4C Setup)

- **Harry (Image Vision):** FastAPI Python microservice for static image analysis.
- **Rockey (Text/RAG):** Node.js text-processing endpoint for the Triage Chatbot.
- **Christo (Video/Temporal & GenAI):** Video frame processing service and the Pet Memory GenAI background worker.
- **Richard (System Architect):** React Native Frontend, Node.js Engine Room, MongoDB schemas, Personalized Nutrition algorithm.



The Symphony: How SmartPet AI Flows in V1

Imagine a single user journey during the V1 launch. This is how the code and the team's models talk to each other in perfect harmony:

1. **The Intake (Richard's Domain):** A user downloads the React Native app, signs up, and creates a **pet profile for their puppy**, "Rex". Richard's frontend sends this payload, and it is saved safely in MongoDB.
2. **The Crisis (Rockey's Domain):** At midnight, Rex starts coughing. The user opens the clean chat interface. They type: *"Rex is coughing badly."* Rocky's backend code intercepts this, grabs **Rex's age and breed from the database(From the Pet profile developed by the user)**, merges it with the text, and feeds it into the RAG model. The bot instantly replies with grounded, safe advice based strictly on Rex's profile.
3. **The Visual Evidence (Harry's Domain):** The user notices Rex also has a weird red rash on his ear. Instead of the chat box, they navigate to the **Skin & Eye Scan** (Photo Scanner) on the dashboard. They snap a photo. Richard's frontend detects the image file and routes it directly to Harry's FastAPI microservice. Harry's computer vision model scans the pixel grid and returns a payload to the backend: `{ "condition": "ear_mites", "confidence": 0.88 }`.
4. **The Movement Test (Christo's Domain):** The user mentions Rex is also walking weirdly. They click the **Gait & Motion Analysis** (Video Scanner) button and upload a quick 5-second video clip. Richard's frontend ships the video to Christo's temporal vision service. His model watches how the frames change over time, flags a slight right-hind leg limp, and sends that status back.
5. **The Payoff & Loop (Richard's Domain):** Now, Richard's **Personalized Nutrition Engine** kicks in. It sees Rex is a puppy, has a diagnosed ear mite skin condition from Harry, and a leg strain from Christo. The algorithm automatically filters the store inventory to hide chicken products (if allergic) and prominently displays puppy-safe anti-inflammatory salmon kibble and joint supplements.
6. **The Wrap-Up (Christo's Domain):** At the end of the month, Christo's GenAI pipeline reads all these stored health events from the database and compiles them into a beautiful, shareable "Health Summary Card" for the owner.



The 4C V1 Specification Document

Component 1: Core App Frontend, Backend & Personalized Nutrition

- **Engineer:** Richard (System Architect)
- **What it does in V1:**
 - **The App Shell:** A manual, hand-coded React Native mobile interface featuring a clean, tab-based navigation layout (Dashboard with dedicated Scanner buttons, Unified Chat Gateway, and E-commerce Storefront).
 - **The Engine Room:** A robust Node.js/Express server mapped to MongoDB Atlas. It handles secure user sign-ups, JWT login validation, and the standard CRUD operations for Pet Profiles (Name, Age, Breed, Allergies).

- **Personalized Nutrition AI (The Logic Engine):** A pure algorithmic script running on the backend. It uses structural conditional branching (if/else matrix matching) to scan incoming AI diagnostic payloads (from Harry and Christo) and automatically generate an inventory query filter. If a pet has a skin rash, it queries MongoDB to return items tagged with Omega-3 fatty acids while strictly filtering out any items containing the pet's logged allergens.
- **E-Commerce Pipeline:** A simple cart system that compiles chosen product objects into an array, serializes them, and drops them into a clean, URL-encoded string that launches WhatsApp to finalize the transaction with the store owner.

Component 2: Vet Triage RAG Chatbot

- **Engineer:** Rockey (ML Engineer - Text/RAG)
- **What it does in V1:**
 - **The Text Core:** A dedicated text-processing API endpoint inside the Node.js environment. It does not deal with files, cameras, or media attachments.
 - **The V1 RAG Pipeline:** When a query hits his endpoint, Rockey writes a script to look up the sender's active Pet Profile ID from MongoDB. He retrieves the raw text parameters (e.g., *"Rex, German Shepherd, 2 years old, allergic to beef"*). He combines this exact string with the user's message into a strict, hard-coded System Prompt template and pipes it to the Gemini/OpenAI SDK. This guarantees the language model is mathematically boxed into referencing the pet's profile metadata, preventing arbitrary medical hallucinations.

Component 3: Static Image Diagnostic API

- **Engineer:** Harry (ML Vision Engineer)
- **What it does in V1:**
 - **The Sandbox:** An isolated, standalone Python web service built using FastAPI.
 - **The Image Core:** A single [POST /api/vision/analyze](#) endpoint that accepts a raw static image file uploaded from the React Native Skin & Eye Scanner.
 - **V1 Logic:** Harry utilizes an open-source, pre-trained image classification network (like a MobileNet or ResNet backbone via PyTorch) fine-tuned on basic veterinary datasets of common superficial skin anomalies or dental issues. The model processes the spatial pixel values of a single frame and outputs a structured JSON response containing the most likely condition string along with a confidence metric.

Component 4: Video Gait & Health Summary Engine

- **Engineer:** Christo (Applied AI & Temporal Vision Engineer)
- **What it does in V1:**
 - **The Motion Core:** A standalone processing service built to handle [.mp4](#) video files up to 5 seconds long uploaded via the Gait & Motion Analysis scanner.
 - **V1 Video Logic:** Instead of complex continuous video streaming, Christo's V1 architecture breaks the incoming video file down into sequential, static

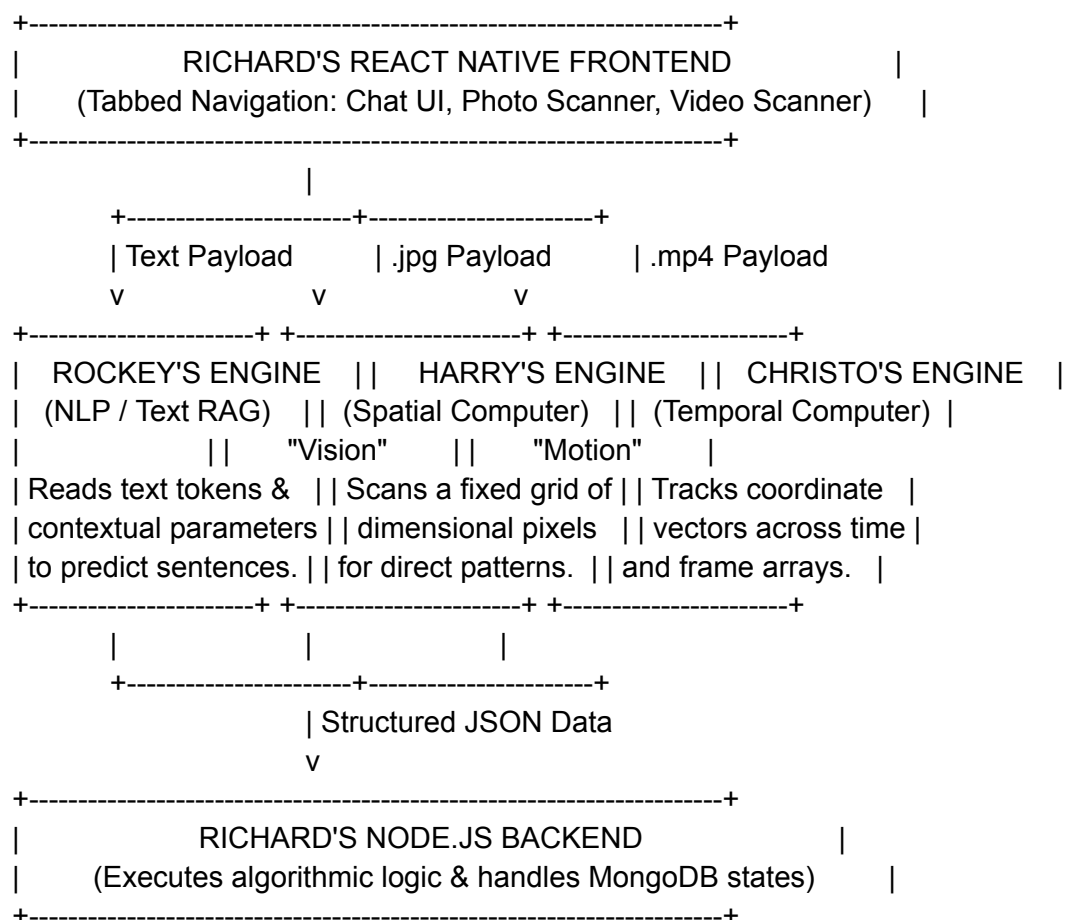
frame arrays. He runs these frames through a spatial tracking framework (like MediaPipe) to map coordinate points on the pet's joints over the video's timeline. By calculating changes in the height-variance curves of those tracking points across successive frames, his script mathematically flags systemic deviations—identifying basic structural gait anomalies like a distinct limp or severe head tilt.

- **Pet Memory AI:** A secondary text-based background worker. It queries the user's historical monthly timeline in MongoDB (total items ordered, logged triage alerts) and structures it into a summary template. It uses an LLM to rewrite this data into a friendly, shareable milestone narrative card.

Why They Seem Identical to Users, But Are Massively Different Inherently

When a user interacts with the app, they just see text, photos, and videos generating intelligent responses. They might think it's all the "same AI." Here are the underlying engineering realities that separate these domains:

Plaintext



The Inherent Architectural Divide:

- **Rockey's Text vs. Harry's Pixels:** Rocky's model lives in a world of language tokens, semantic weights, and string context. It cannot interpret an image because it

cannot compute spatial arrays. Harry's model lives completely inside a static, non-moving matrix of color values (R, G, B numbers across rows and columns). It checks textures and color changes in a single slice of time; it has zero concept of language, sentences, or history.

- **Harry's Photos vs. Christo's Videos:** Harry's model assumes the world is perfectly still. If you give Harry a blurry video frame, his model breaks because the spatial textures melt. Christo's model, however, relies on change. It cannot diagnose a skin allergy from a single photo because it needs to observe how coordinates transform relative to each other over a multi-frame timeline (T). It measures velocity, angles, and physical mechanics across multiple dimensions over time.
- **The AI Components vs. Richard's Engine Room:** Rockey, Harry, and Christo are building highly specialized sensors. They create independent tools that generate raw data points. Richard's engine room is what transforms that raw data into business value. Their models don't know that the store has a sale on salmon kibble; Richard's backend is what consumes their machine learning outputs and executes the actual e-commerce application rules.

SmartPet AI (V1) – API Integration & Development Protocol

To ensure a seamless integration phase and maintain decoupled development streams, all team members must adhere to the following architectural guidelines before commencing implementation. These protocols are designed to eliminate cross-origin errors, prevent payload mismatches, and allow parallel development across the React Native client and our AI microservices.

1. Schema Standardization (The "Contract-First" Architecture)

To prevent parsing failures on the client side, backend and frontend development must be completely decoupled. Before any service logic is written, the Lead Architect (Richard) and the respective microservice owner must establish a strict API Contract.

- **Protocol:** We will define the exact JSON payload structures, HTTP status codes, and data types (e.g., ensuring `confidenceScore` is a float, not a string) in a shared repository document.
- **The "Why":** If the React Native client expects `{ "status": "success", "message": "Feed him salmon." }` and the Node.js API returns `{ "responseCode": 200, "data": { "text": "Feed him salmon." } }`, the application state will crash. Keys, casing (camelCase), and nesting structures must be locked in writing before coding begins.

2. Independent Endpoint Validation

No microservice will be integrated into the mobile client until it has been proven stable in an isolated environment.

- **Protocol:** Feature branches will not be merged into the staging environment until the developer provides a working Postman collection or a reproducible `cURL` command.
- **The "Why":** If the image classification model fails on the frontend, we must immediately know if it is a React Native networking error or a FastAPI server fault. By requiring isolated API validation, we ensure that only stable, parseable data enters the mobile ecosystem.

3. Local Environment & CORS Configuration

Because we are orchestrating a Node.js core alongside multiple Python-based AI engines, running disparate local servers will trigger Cross-Origin Resource Sharing (CORS) blocks and port collisions.

- **Protocol:** All local environments must strictly adhere to the following port mappings. Ensure your respective server frameworks (Express, FastAPI) are configured to accept CORS preflight requests from the mobile emulator.
 - **React Native Client (Richard):** Mobile Emulator/Expo
 - **Core Backend / RAG API (Rockey):** `localhost:5000`
 - **Static Vision FastAPI (Harry):** `localhost:8000`
 - **Temporal Vision Engine (Christo):** `localhost:8001`

4. Client-Server Decoupling via Data Mocking

The frontend UI development will not be bottlenecked by the machine learning training cycles or database configuration times.

- **Protocol:** The React Native application will be built immediately using hardcoded, mocked JSON payloads that perfectly mirror the agreed-upon API Contracts (Rule 1).
- **The "Why":** By isolating the frontend state management using mocked data, Richard can complete the UI components, navigation stacks, and error handling in parallel with the AI development. Once Rockey, Harry, or Christo deploys a stable local endpoint, the mock payload is simply hot-swapped for the active `localhost` URL via `.env` variables, instantly connecting the systems with zero structural refactoring.